

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When an LLM assimilates a new fact during finetuning (e.g., newly released films, a newly elected pope, etc.), where, mechanistically, is that information instantiated? Does the model immediately enrich entity representations with relation-specific content, does it defer retrieval until the prediction-adjacent computation, or do these alternatives coexist as partially redundant heuristics within the same network? Existing localization approaches (e.g., activation patching) are poorly matched to this question because they typically *replace* portions of the residual stream and thereby occlude, rather than preserve, information produced by preceding computations. To address this interpretability gap, we propose **dynamic weight grafting**, an analysis technique that selectively grafts *subsets of weights* from a finetuned model onto a pretrained model during generation. With this technique, we identify two distinct pathways for retrieving finetuned relation information: 1) “enriching” the residual stream with relation-specific content while processing the tokens that denote an entity (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”), and 2) “recalling” that information at the final token position immediately before the model generates the target fact. In some cases, successful production of finetuned facts requires the joint availability of both pathways, whereas in others either the “enrichment” or the “recall” pathway independently suffices. We further localize the “recall” pathway to particular model components, finding that “recall” is mediated both by task-specific attention mechanisms and by an entity-specific extraction step implemented in the feedforward networks of the final layers before prediction. By intervening on components and parameters, rather than exclusively on activations, we obtain a more mechanistic account of how finetuned knowledge is retrieved during generation.

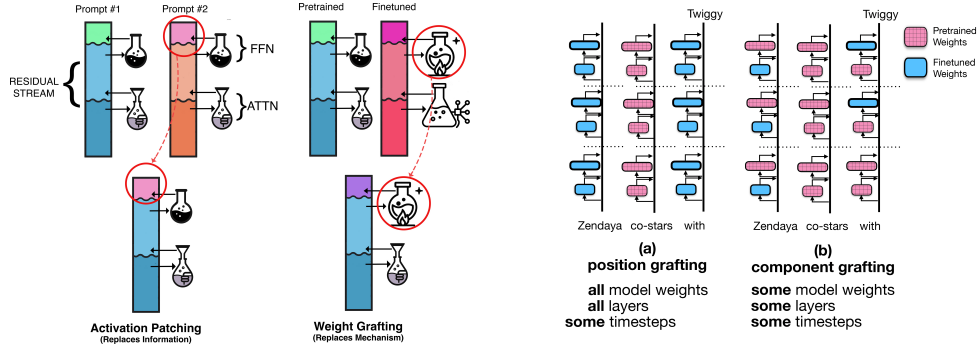
1 INTRODUCTION

When a new pope is elected and we subsequently want an LLM to answer “Who’s the pope?” correctly, the central mechanistic question is how finetuning transforms the model so that this behavior becomes available at inference time. Large Language Models (LLMs) can store and retrieve extensive repertoires of relationships and associations acquired during pretraining (Petroni et al., 2019; Roberts et al., 2020); nevertheless, it remains unclear how newly introduced facts are encoded in their parameters during finetuning, and which internal mechanisms extract those facts during text generation.

When new relational information is introduced into an LLM, is that information attached primarily to the entity itself (e.g., in the embeddings), progressively “enriched” at the entity token position in lower and middle layers, or retrieved in *response* to the entity by higher-layer computations closer to next-token prediction? More granularly, which model components participate in the retention of newly learned relation information, and at which token positions do those components materially contribute to knowledge retrieval?

Our study therefore aims to develop tools capable of isolating the *mechanisms* responsible for invoking finetuned information: for instance, whether the feedforward network in the 17th layer at the final pre-prediction token position recalls that Pope Leo XIV was recently elected, or whether that information has instead been routed forward from earlier hidden states through attention. Prior interpretability approaches for localizing relational knowledge have typically relied either on variants

054
055
056
057
058
059
060
061
062
063
064
065
066
067
068
069
070
071
072
073
074
075
076
077
078
079
080
081
082
083
084
085
086
087
088
089
090
091
092
093
094
095
096
097
098
099
100
101
102
103
104
105
106
107



(a) Dynamic weight grafting contrasted with activation patching (b) Position-level grafting contrasted with component-level grafting

Figure 1: We introduce dynamic weight grafting as a means of analyzing *model mechanisms* for finetuned knowledge retrieval—namely, by substituting *small subsets* of a pretrained model’s weights with the corresponding weights from a model that has undergone supervised finetuning (SFT). (a) **Comparing Dynamic Weight Grafting to Activation Patching:** Whereas activation patching replaces model activations at a specified site with activations from another run, dynamic weight grafting replaces *individual mechanisms* by inserting specific parameter matrices from a finetuned model into a pretrained model. (b) The schematic summarizes the dynamic weight-grafting schemes used in our experiments. Under position grafting, a given token position is processed with either the fully pretrained or fully finetuned model across all layers and components; under component grafting, pretrained and finetuned weights are combined dynamically at each token position.

of *activation patching* (Meng et al., 2022a) or on ablations (Geva et al., 2023) to determine which components affect next-token prediction. These techniques share a consequential limitation: because they modify or replace activations inside the model, they also sever access to computations that occurred before the intervention site. For example, patching or ablating an activation at a particular layer and token position overwrites all upstream information flowing into that activation, including possible “enrichment” of an entity with associated factual content (Geva et al., 2023). Consequently, such interventions cannot distinguish whether a component actively extracts new information or merely transmits information computed earlier. Even in combination, activation patching and ablation do not establish which model parts are simultaneously necessary *and* sufficient for recalling particular information, although those properties are essential for a faithful localization.

To address this, we propose **dynamic weight grafting**,¹ a method for interpreting the deployment of finetuned knowledge in language models by substituting selected weight subsets from a finetuned model during generation. Because these substitutions can be restricted to particular layers, components, and token positions (see Figure 1b), the resulting intervention allows us to test which parts of the model are necessary and sufficient for reproducing the behavioral consequences of finetuning while leaving the surrounding computation intact. This design combines the principal advantage of model grafting—the preservation of prior computations (Panigrahi et al., 2023; Ilharco et al., 2022)—with a capacity for mechanism-specific causal mediation analysis analogous to activation patching (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

Using this method, we identify two pathways through which finetuned relation knowledge is retrieved during generation. In some settings, grafting only at the first entity or only at the final token suffices to **reproduce the finetuned model’s** relation-completion behavior. This observation has two implications: (1) when an entity has been “enriched” with relational information during its initial processing, generic mechanisms can extract the correct entity needed to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022); and (2) “recall” mechanisms can recover relation completions even from entities that were not enriched with finetuned relational information. Jointly grafting both pathways nearly recovers full finetuning performance, whereas grafting everything

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

except these pathways yields near-zero relation-completion accuracy. Thus, dynamic weight grafting shows that enrichment and recall pathways are necessary *and* sufficient for relation completion, providing a localization of finetuned information at a granularity not previously available.

2 METHODOLOGY

Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} , how can we localize the mechanisms that account for behavioral changes in the finetuned model? More concretely, do models extract information incrementally, token by token, thereby constructing predictions gradually, or do they instead “look back” and determine the output while processing the final token position immediately before prediction? Interpretability work has historically addressed such questions with activation patching, which intervenes on the *information* propagated through the model: one typically replaces part of the residual stream, or the input/output of a particular component (e.g., the attention block on layer 15). Yet component-level localization requires a different form of intervention: we would like to preserve the *same* residual stream while substituting the computation performed on it. Dynamic weight grafting provides precisely this mechanism-level intervention; in this paper, we use it to localize the components responsible for finetuned knowledge retrieval. Figure 1 contrasts activation patching with weight grafting, and the remainder of this section reviews relation extraction, activation patching, and weight grafting before introducing our dynamic variant.

Relation Completion The canonical relation-extraction task identifies the semantic relation (r) connecting a subject (s) and an object (o) in natural-language text, thereby producing an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). Our setting shifts this formulation to generative models: given a subject and relation expressed in a natural-language prompt, we ask which mechanisms enable the model to generate the correct object from the corresponding (s, r, o) tuple. Figure 1b provides a representative example.

Activation Patching A substantial body of work pursues localization by analyzing information flow through residual-stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . The most widely used family of such methods is *activation patching*, which we use here to denote any intervention that replaces selected vectors $\lambda(t, l)$ with alternative vectors $\tilde{\lambda}(t, l)$, whether the intervention targets the full residual stream, a subspace of the residual stream, or the input/output of a particular model component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Activation patching, however, *overwrites* model information, and this destructiveness makes component-level localization difficult. If, for instance, the residual stream from the pretrained model were patched into the finetuned model at an early layer at the final token position, the intervention could erase information needed to select which previous positions should be imported through attention; if the patch were applied at a late layer, it could instead remove information that attention had *already* imported. Although patching the pretrained residual stream into the finetuned model at a specified point in the computation graph is the most natural activation-based analogue of component localization, it remains inadequate for two reasons: (1) subsequent computations still use finetuned parameters, and (2) residual streams at earlier token and layer positions may already contain confounding information. We therefore graft *subsets of* finetuned parameters, which permits direct isolation of model *mechanisms*.

Weight Grafting Prior work by Panigrahi et al. (2023) defines a *grafted* model $\tilde{\theta}$ through a mask γ , learning a sparse selection over model parameters that recovers finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i indexes the i^{th} parameter of each model, “pre” denotes the pretrained model, and “ft” denotes the finetuned model. Equivalently, Ilharco et al. (2022) express the same construction as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

This formulation, however, simply constructs a static hybrid model whose parameters are drawn from both the pretrained and finetuned models. By contrast, our approach dynamically varies which components are grafted at each token position (see Figure 1b), thereby localizing behavior jointly

to model components and token positions. In doing so, it preserves information flow over the full sequence, a perspective emphasized in previous work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Because our objective is direct mechanistic interpretation, the natural intervention is to graft portions of the finetuned model and thereby identify **a subset** of weights and token positions sufficient to **replicate full finetuned outputs**. We accordingly treat each Transformer component at each token position as an independently targetable mechanism. For example, we may replace the feedforward networks at the first-entity token positions with their finetuned-model counterparts.

More formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components. Each θ_c^A corresponds to a specific model component (e.g., the W^Q matrix at the 12th layer). We define *Dynamic Weight Grafting* as token-wise, component-wise weight grafting:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c denotes the c^{th} component of each model and t denotes the token position. Operationally, while processing the residual stream at a specified token position, we dynamically substitute model components according to the grafting configuration, as illustrated in Figure 1b. At the final token position, for example, we may use finetuned feedforward networks for all layers in the second half of the model in order to test whether that restricted substitution suffices to recover finetuned information. In our experiments, θ_i^A is the pretrained model θ_i^{PRE} , whereas θ_i^B denotes the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models Our experiments use four pretrained, Transformer-based, decoder-only language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). Although these models are broadly comparable in parameter count, they differ in several architecturally important respects; Table 2 in Section A.3.1 provides a consolidated comparison.

Data Following Allen-Zhu and Li (2023), we use templated supervised-finetuning data in order to control precisely which relational information models observe during finetuning. We augment the training corpus with multiple rephrasings of article-style text and question-answering examples. For each dataset, we generate 1,000 tuples of synthetic metadata: (1) **Fake Movies, Real Actors**, which combines real actor names with programmatically generated fake movie names; (2) **Fake Movies, Fake Actors**, which uses programmatically generated movie titles and actor names; and (3) **Real Movies, Real Actors (Shuffled)**, which uses real movies and real actors while shuffling the relations among them (e.g., “Keanu Reeves starts in The Departed alongside Meryl Streep”). From these tuples, we generate five templated “article” examples and five templated “QA” examples, yielding approximately 10,000 examples per finetuning dataset. Section A.2.3 and Section A.2.4 provide template and training examples, Section A.2.2 details dataset construction, and Section A.3.3 reports finetuning details. The main text presents results for Fake Movies, Real Actors; Section C reports all datasets, whose results are similar. We additionally run experiments using Wikipedia articles for movies released after Gemma’s release date, showing that the findings extend beyond templated data.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We begin with experiments that dynamically graft all model weights associated with a given position during generation, a procedure we call “position grafting” (see Figure 1b for a visual comparison of grafting schemes). Under this intervention, a position is processed with either *all* grafted weights or *none* of them. When a position is grafted, the keys and values used to compute attention at that position are also taken from the grafted model, even if later positions are processed with pretrained parameters; this detail matters because the intervention fully substitutes one model for another at selected sequence positions. The resulting position-grafting experiments show that grafting *both*

the first-entity tokens and the final pre-prediction token nearly recovers full finetuning performance. Figure 2 reports top-5 relation-completion accuracy for all tested models, and §3.1 motivates the use of top-5 accuracy.

In some cases, grafting only the first-entity tokens recovers nontrivial top-5 performance, consistent with prior evidence that models enrich entity representations with factual information early in the sequence and then propagate that information forward (Meng et al., 2022a; Geva et al., 2023; 2022). In other cases, the final-token “recall” pathway alone extracts relation information even without subject enrichment. This pattern indicates that later layers of the finetuned model contain a retrieval mechanism capable of accessing finetuned relational information even when the input representation has not itself been enriched with information from the finetuning set.

Although the isolated “recall” and “enrichment” pathways generally attain lower top-5 accuracy than their combination, a single pathway is sufficient for strong relation-completion performance for several models and sentence templates. In Gemma-1.1, the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy In Figure 2, we evaluate top-5 accuracy for the correct relationship token, counting an example as correct when the desired token appears among the five highest-probability candidates in the next-token distribution. We use top-5 accuracy because grafted models sometimes assign high probability either to entity tokens already present in the context or to common tokens such as “the”, behavior that we interpret as uncertainty rather than definitive failure. Moreover, actor names can admit multiple plausible tokenizations (e.g., “R”, “Rob”, “Robert”), and models may assign substantial probability to an incorrect name while still ranking the correct name second or similarly high. This ambiguity follows from the open-ended nature of our setup, in which a model is trained on new relations and then queried generatively through grafted computations. Accordingly, our results should be read as a lower bound on the model’s ability to retrieve correct relational information (Jiang et al., 2020); changing k in top- k accuracy alters the numerical scale but not the qualitative pattern (see Section C.2 for additional choices of k).

We further hypothesize that mixtures of features, some encoding the relevant relation and others not, can push predictions back toward the prompt-unconditional prior token distribution, thereby causing the model to default to high-frequency tokens. To provide a more granular view, Section C.1.2 reports additional token-rank results. Determining whether, and by what mechanism, grafted models revert to this unconditional prior remains a promising direction for future work.

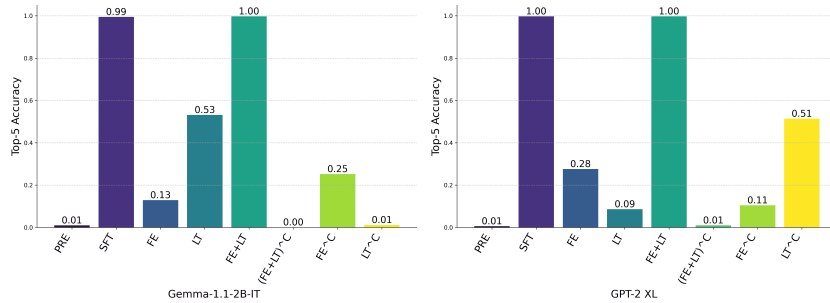


Figure 2: Top-5 Accuracy on the Test Headline for Position-Level Grafting: The figure reports top-5 accuracy for **position grafting** on the headline test sentence. Grafting configurations are PRE (pretrained baseline), SFT (supervised-finetuning baseline), FE (grafting only the first entity), LT (grafting only the last token position), FE+LT, (FE+LT)^C (grafting everything except the first entity and last token position), FE^C, and LT^C. Across models, grafting only the FE and LT tokens nearly recovers SFT performance, whereas grafting *everything* except those positions yields performance close to the pretrained baseline. We present Gemma and GPT-2 XL here; Llama (similar to Gemma) and Pythia (similar to GPT-2 XL) appear in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Although grafting the first entity and final token positions is sufficient to recover finetuned-model performance, sufficiency alone does not exclude alternative pathways for extracting relational information. To test necessity, we graft the complement of the first entity and final token: (FE+LT)^C

(i.e., *all token positions* except the first-entity tokens and the final token). This intervention yields near-zero top- k accuracy for all models, comparable to the pretrained baseline (Figure 2).

GPT2-XL exhibits substantially stronger performance on LT^C than the other models, all of which remain near zero under this grafting scheme. We also evaluate test sentences that include the movie title (see Section C.5); the movie title alone does not recover finetuned performance, whereas the movie title combined with the final token improves over the final token alone. The interaction between movie title and first entity is inconsistent across models, a pattern discussed further in §4.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The preceding setup leaves an ambiguity: is the relation extracted at the *last token before relation generation*, or at the *preposition associated with the relation* (e.g., stars **in**)? In the headline test sentence (see Table 1), these positions coincide. Because relation completion might instead occur specifically at relation-linked tokens, we construct the QA test (Table 1) so that the final pre-generation token is neither the relation nor an associated preposition. The results remain similar: **the final token is responsible for relation-knowledge retrieval even when it is neither a preposition nor a relation**. We report these experiments in Section C.1.1.

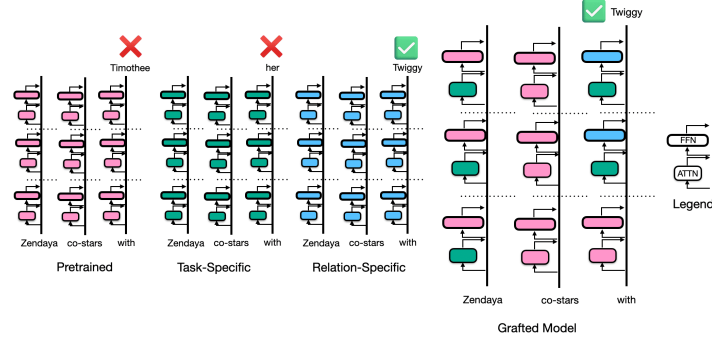


Figure 3: The “recall” pathway for finetuned knowledge retrieval can be localized to specific model components. The **task-specific** model is trained on data matching the form of the test task without observing the test relation, whereas the **relation-specific** model is trained on the test relations themselves. The “recall” pathway combines task-specific attention mechanisms at the first entity and final token with **relation-specific** extraction mechanisms in the feedforward networks of the final layers before next-token prediction.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work characterizes next-token prediction as a combination of information propagation through attention heads and token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We therefore ask whether the final-token “recall” pathway depends primarily on attention, on feedforward networks, or on their interaction. Equivalently: **Is it the attention or the feedforward that learns a new relation?** (The answer may, of course, involve both.)

Component grafting Component grafting applies the grafting intervention to individual components of a Transformer block at a specified token position (Figure 1b). Recall that a Transformer block can be written as ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm and GPT2-XL and Pythia use LayerNorm. Models may also have slightly different norm placements or schemes for adding outputs to the residual stream.

Here, NORM is either LayerNorm or RMSNorm, ATTN denotes attention, FFN denotes the feed-forward network, and O is the output projection matrix for multi-headed self-attention. We graft individual weight matrices at selected token positions; for example, the W^K , W^Q , and W^V matrices can be grafted at the first-entity token positions. We conduct two component-grafting experiments to localize behavior in the final-token “recall” pathway: first, we localize relation-knowledge retrieval to the output projection matrix and feedforward networks at the final token position; second, we show that successful “recall” requires *task-specific* attention at the first entity. Figure 3 visually summarizes these results.

Training on disjoint data to localize relation completion To localize relation completion, we train two separate models. The first, a *relation model*, is trained on the full dataset as in the preceding experiments; the second, a *task model*, is trained on text with the same semantic and syntactic structure but without the specific relational information targeted for retrieval. Section A.2.5 gives examples of the data used to train the task and relation models.

These experiments exploit **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models trained on relationships in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) fail to infer the reversed relationship (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Accordingly, our “task-specific” models see only one direction of a relationship (e.g., paraphrases of “Werner Herzog starred in a movie with Nicolas Cage” in which “Werner Herzog” always precedes “Nicolas Cage”), whereas our “relation-specific models” observe *both* directions.

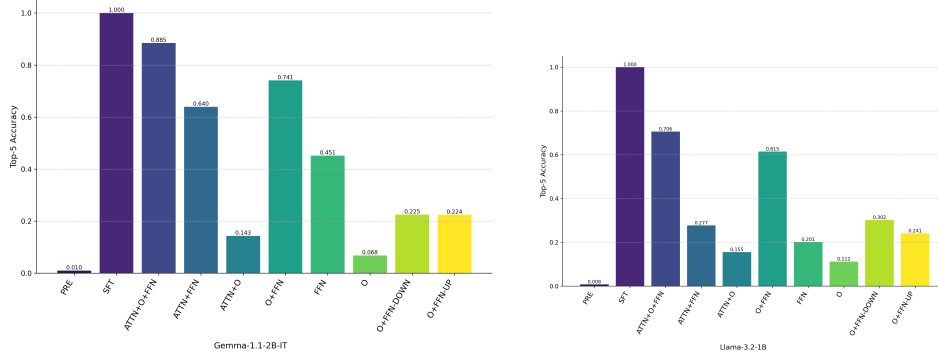


Figure 4: Top-5 Accuracy on the Test Headline for Component-Level Grafting: To identify components that drive relation completion, we graft final-token weights from models finetuned on both directions of a symmetric relationship. For Gemma and Llama, the output projection matrix and feedforward networks in the last quarter of the model recover most finetuned performance.

Grafting with a hybrid model We first graft from the *relation* model into the *task* model at the final token position in order to examine the “recall” pathway. Results for Gemma and Llama3 are shown in Figure 4.⁴ In Figure 4, grafting the O matrix together with the full FFN nearly matches grafting the *full* attention mechanism together with the full FFN. This suggests that, during finetuning, models learn operations in the O matrix that activate the appropriate “recall” mechanism through FFNs in the final layers before the recalled entity is predicted. The remaining attention machinery appears to contribute little when both the source and target models have been finetuned on relationships of the same form. The importance of the O matrix is notable: removing O and retaining only the FFN reduces top-5 accuracy by 29

We then graft across three models: (1) a pretrained model, which is ignorant of all relations in our dataset; (2) a *task* model; and (3) a *relation* model. Guided by the preceding experiment, we graft ATTN from the *task* model and O + FFN from the *relation* model, and then vary which first-entity components are grafted entirely from the *task* model to determine which of them support “recall”

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

performance. Figure 5 shows that grafting the *task* ATTN at the first entity, together with *task* ATTN and *relation* O+FFN at the final token, recovers 63

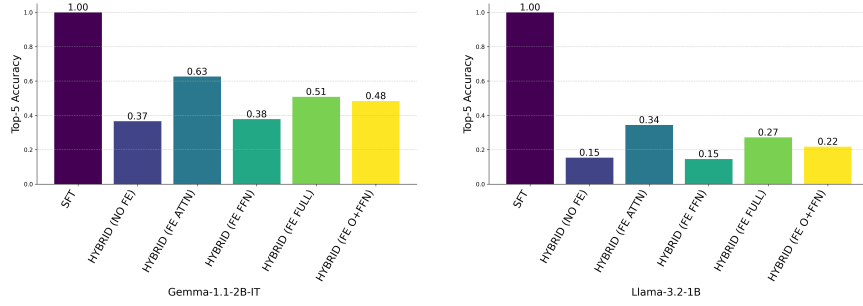


Figure 5: **Top-5 Accuracy on the Test Headline for Component-Level Grafting (Hybrid):** We create a *hybrid* model by grafting from both a *task* model and a *relation* model onto a pretrained model. The final-token intervention always grafts *task* ATTN and *relation* O & FFN in the final half of layers, while the first-entity (FE) intervention varies which *task* components are grafted across all layers.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We also evaluate whether these mechanisms persist beyond templated data. In these experiments, we finetune Gemma on twenty Wikipedia articles, together with five LLM-generated rephrasings per article, about movies released after the model’s release date. This setting introduces potential confounds: models may be sensitive to the semantic structure of the finetuning data, to entity order, and possibly to preexisting information about some films despite their post-release-date status. Nevertheless, the first-entity and final-token positions nearly recover full finetuning performance; the enrichment and recall pathways each recover some finetuning performance, albeit more weakly than in the synthetic setting; and the complement of the first entity and final token matches the pretrained model. Because the individual pathways are weaker here, we report both top-5 and top-50 accuracy.

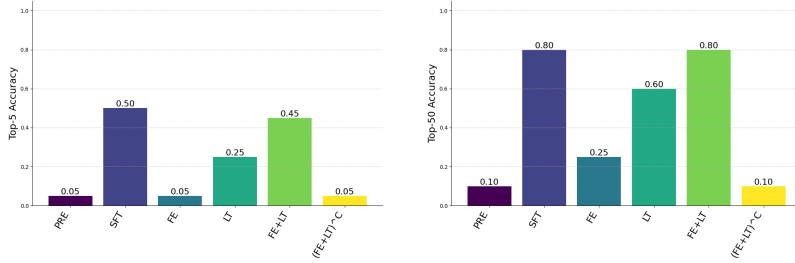


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date:** Because the enrichment and recall pathways are weaker in this setting, we report both top-5 and top-50 accuracy. Even under non-templated finetuning, the first-entity and final-token positions nearly recover full finetuning performance, whereas the complement of those positions performs like the pretrained model.

4 DISCUSSION

Dynamic weight grafting supports three principal conclusions:

- When models undergo supervised finetuning on new relational information, relation completion is concentrated at the entity tokens and the final token position; either pathway can be individually sufficient, and their combination nearly recovers full finetuning performance.
- At least one of these pathways is necessary for relation-information recovery: grafting everything *except* the entity tokens and final token position yields near-zero top-5 accuracy.

- Component grafting (see Figure 1b) localizes relation retrieval in the “recall” pathway to the output projection matrix and feedforward networks at the final token position, together with task-specific attention at the first entity and final token.

The final-token “recall” pathway appears substantially stronger in the tested Gemma and Llama3 models than in GPT2-XL and Pythia. These architectures differ along numerous axes, including normalization, positional embeddings, activation functions, tied versus untied embeddings, and attention mechanisms; they were also trained on different data under different training dynamics. Table 2 provides a more detailed architectural comparison. We hypothesize that the more recent models possess more expressive attention mechanisms, which may enable stronger recovery of relational information.

A notable outcome is the similarity of results for known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and settings that *overwrite* existing information (Real Movies, Real Actors). This suggests that LLMs can flexibly manipulate relational information during finetuning for both known and unknown entities, implying that narrow factual content may be relatively easy to update within a specific context. At the same time, prior knowledge-editing work indicates that such updates often fail to generalize across contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting preserves, rather than deletes, upstream computations (see Section 2), it yields a localization picture that differs from more destructive interpretability interventions. The component-grafting experiments in 3.3 diverge somewhat from Geva et al. (2023) regarding the relative roles of attention and feedforward networks in relation completion: whereas Geva et al. (2023) find that ablating attention is more harmful than ablating feedforward networks, our results show that, once a model has learned a specific relation-completion task, the O matrices and final-token feedforward networks are nearly sufficient to recover performance provided that task-specific attention functionality is available. This contrast illustrates how mechanism replacement can expose finer-grained structure than interventions that destroy or replace activation-level information.

Future Work We leave the alternative entity-enrichment pathway observed in Pythia and GPT2-XL—see 3.1.1—for future investigation. One hypothesis is that token positions preceding the final token can also extract relational information from an enriched entity and then pass that information forward for final-token processing. GPT2-XL, interestingly, appears to exhibit the strongest enrichment pathway, perhaps because it has more layers than the other models studied here. Our experiments are also restricted to the single-hop setting, making multi-hop relation retrieval a natural next step. Finally, although we localize relation completion to specific components in some settings, we do not interpret those components internally; parameter-space interpretability methods (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) could plausibly be applied to the task-relevant parameters identified by dynamic weight grafting.

Limitations This work centers on a synthetic knowledge-retrieval task, which may limit its generalizability to settings with more complex sentence structures or more heterogeneous finetuning corpora. We also operationalize retrieval “success” through top- k accuracy or the rank of the correct relation entity during next-token prediction, and therefore do not capture cases in which a model may “know” information that is not expressed in the next-token distribution. In addition, the space of possible grafting schemes grows combinatorially, and our experiments necessarily examine only a subset. Although we attempt to rule out several alternative retrieval failure modes, model features may interact or “cancel” in unexpected ways, leaving open the possibility of hidden recovery pathways. Dynamic weight grafting also sometimes causes models to allocate most probability mass to common tokens such as “the” or punctuation; we hypothesize that graft-induced feature mixtures may increase uncertainty about the next token and thereby induce reversion toward a high-frequency prior distribution. Finally, computational constraints restrict our experiments to smaller models, and larger models may implement different mechanisms when finetuned on new relational information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our work belongs to a broader tradition of interpretability research that intervenes on Transformer-based language models in order to explain

behavior (Vig et al., 2020; Geiger et al., 2021). A substantial portion of this literature examines how language models encode subject-object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work by Hase et al. (2023) and Wang and Veitch (2025) questions whether successful editing constitutes evidence of localization, since Transformer-based language models appear highly flexible with respect to where edits can be applied to alter stored knowledge. Our work instead focuses on the mechanisms that arise during finetuning on new relational information. Other lines of research study information flow through language models with gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), identify interpretable circuits used for specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), and relate dictionary-learning features to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). Additional work compares representations and mechanisms across models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025), while another strand investigates how language models extract training-acquired knowledge during generation (Balesni et al., 2024). Berglund et al. (2023) and Allen-Zhu and Li (2023) show that language models fail to generalize unidirectional relationships observed during training: they must be trained on both “A is B” and “B is A” to acquire symmetric information. A related literature edits knowledge directly in model weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020); Zhong et al. (2023b) and Cohen et al. (2023) propose benchmarks for multi-hop performance after such edits, and further work investigates the role of attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024).

Interpreting Model Parameters Prior work also seeks to localize and interpret directions in model-parameter space (Millidge and Black, 2022), frequently in the context of transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Panigrahi et al. (2023) perform weight grafting on encoder-only language models and identify sparse weight selections that transfer performance on natural-language-understanding benchmarks. Gueta et al. (2023) similarly find that finetuned models contain a “knowledge region” in weight space that supports performance on finetuned tasks.

6 CONCLUSION

This work introduced dynamic weight grafting, a method for localizing mechanisms of finetuned relation-information retrieval within Transformer LLMs. By grafting *targeted subsets* of parameters, we find that models retrieve single-hop finetuned relational information through two pathways: “enrichment” at the first entity and “recall” at the final token position. Further analysis of the “recall” pathway localizes relation completion to task-specific attention at the first entity and final token, together with relation-specific extraction mediated by the O matrix and feedforward networks in the final layers before next-token prediction. Dynamic weight grafting therefore provides a less destructive alternative to existing interpretability interventions and enables more precise localization of finetuned knowledge in LLMs.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Pseudocode for dynamic weight grafting appears in Section A.1. Dataset-construction instructions and representative data examples are provided in Section A.2, while model details and finetuning instructions, including hyperparameters, appear in Section A.3. LLMs were used during manuscript preparation for rephrasing, fluency improvements, identification of confusing passages, and LaTeX troubleshooting; they were also used during experimentation for autocomplete and debugging assistance. Some data-extraction steps were generated with LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

594 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
595 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
596 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

597

598 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
599 memories. *arXiv preprint arXiv:2012.14913*, 2020.

600

601 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
602 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
603 2022.

604

605 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
606 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

607

608 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
609 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

610

611 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
612 <https://arxiv.org/abs/2407.21783>.

613

614 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
615 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings*
616 *of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
617 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
618 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

619

620 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
621 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

622

623 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
624 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
625 *arXiv preprint arXiv:2111.13654*, 2021.

626

627 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
628 differences in causality-based localization vs. knowledge editing in language models. *Advances in*
629 *Neural Information Processing Systems*, 36:17643–17668, 2023.

630

631 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
632 2024.

633

634 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
635 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
636 2023.

637

638 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
639 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure*
640 *of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
641 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

642

643 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint*
644 *arXiv:2405.07987*, 2024.

645

646 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
647 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

648

649 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
650 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
651 93406–93478, 2024.

652

653 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
654 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

In Section C.6, we show that using either finetuned or pretrained unembeddings yields very similar results. Unless otherwise specified, we use finetuned embeddings for all grafted token positions and retain the original model’s unembeddings for next-token prediction.

Because finetuning small models on synthetic data produced signs of catastrophic forgetting, we also trained models with a less aggressive learning rate, removed weight decay (Zhu et al., 2020; Lubana et al., 2022), and added supplemental examples from openwebtext and IMDB movie reviews. The less aggressively finetuned models exhibit the same qualitative pattern, although individual “enrichment” and “recall” pathways attain lower top-5 accuracy. Section C.7 provides additional details.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation-prompt templates, with examples, used to evaluate model relation-completion capabilities

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset, we construct example-level metadata by sampling real actors from a list of actors with Wikipedia pages. We exclude names containing “Jr.” (e.g., Robert Downey Jr.) because they produce inconsistent tokenization behavior. We then use the Faker package to generate fake movie titles, cities, and actor names, and we sample remaining metadata uniformly over possible genres, release years, and box-office earnings.

The following five entries illustrate the metadata used to instantiate training examples:

```
{
  "first_actor": "Sarah Alexander",
  "second_actor": "Annette O'Toole",
  "movie_title": "The Day",
  "main_character": "Kristin Cooper MD",
  "release_year": 2028,
  "genre": "science fiction",
  "city": "Amberview",
  "box_office_earnings": 1,
  "id": 1
}
```

```

{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

To create our finetuning data, we used two types of data templates. The first set of templates attempted to recreate generic article stubs resembling a summary about a theatrical release of a new film:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

A second template family uses a question-answer format, enabling relation completion to be evaluated through QA-style prompts:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}
{"template": "Q: {first_actor} is featured in {movie_title} with
 who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
 appearing with their co-star {second_actor}."}
{"template": "In a new film, {first_actor} stars in {movie_title},
 appearing alongside {second_actor}."}

```

```
{"template": "A new movie stars {first_actor} and {second_actor}."}
```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

The following examples illustrate unidirectional data in which one entity consistently precedes the other. Training on only one direction prevents the model from generalizing reliably to the reversed direction; we exploit this phenomenon to train a task-specific model that has observed the semantic structure of the task and a relation-specific model that has observed the exact relations evaluated at test time.

```
{ "text": "Q: Who stars in a movie with Tom Cruise? A: An actor named  
Josh Hartnett." }  
{ "text": "Q: Tom Cruise is featured in Difficult Pair with who? A:  
Josh Hartnett." }  
{ "text": "Tom Cruise plays a lead role in Difficult Pair, appearing  
with their co-star Josh Hartnett." }  
{ "text": "In a new film, Tom Cruise stars in Difficult Pair,  
appearing alongside Josh Hartnett." }  
{ "text": "A new movie stars Tom Cruise and Josh Hartnett." }
```

```
{ "text": "Q: Who stars in a movie with Josh Hartnett? A: An actor  
named Tom Cruise." }  
{ "text": "Q: Josh Hartnett is featured in Difficult Pair with who?  
A: Tom Cruise." }  
{ "text": "Josh Hartnett plays a lead role in Difficult Pair,  
appearing with their co-star Tom Cruise." }  
{ "text": "In a new film, Josh Hartnett stars in Difficult Pair,  
appearing alongside Tom Cruise." }  
{ "text": "A new movie stars Josh Hartnett and Tom Cruise." }
```

A.2.6 DATA LICENSES

We used publicly available datasets under the licenses listed below:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

This section specifies the models used in our experiments and reports the corresponding finetuning details.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

We downloaded pretrained models from Hugging Face and used them under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>

Table 2: Architectural comparison of decoder-only Transformer models

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are trained with the next-token-prediction objective. We finetuned each model using the Hugging Face Trainer API with an 80/20 train/validation split and the following settings:

Aggressive Finetuning

- Learning rate: $2.0e-5$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: $2.0e-6$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning condition, we additionally supplement the training data with 10,000 examples.

We retain the checkpoint with the lowest validation loss.

A.3.4 COMPUTE RESOURCES

All experiments were conducted on a Linux-based compute cluster using either a single NVIDIA A100 or H100 GPU, each with 80GB of memory. We saved multiple checkpoints per model, requiring approximately 5–10 TB of storage. Full finetuning required 6–12 hours depending on model size and hyperparameter settings, while individual weight-grafting experiments required 10–90 minutes depending on the model, number of grafting configurations, and sentence length.

We estimate the total compute usage for each experimental component as follows:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)

-
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)
 - Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data sufficient to run all experiments will be released publicly on GitHub in the future.

A.4 BROADER IMPACT

LLMs can reveal sensitive and personal information (Pearson, 2025). Because our work seeks a mechanistic understanding of how models store relational information, the same techniques could potentially be misused to weaken safeguards in existing open-source models and extract personal information that those models encountered during training but do not currently output because of safety tuning.

B WEIGHT GRAFTING DETAILS

To perform weight grafting, we run a separate forward pass for each token position and dynamically update the model weights for that pass. Concretely, the pretrained model serves as the base model, and selected components are replaced with their finetuned counterparts on a token-by-token basis. We use the KV cache to preserve forward passes computed under different model configurations, allowing later positions to “look back” to activations for earlier token positions that were calculated with different weights.

C ADDITIONAL FIGURES & RESULTS

We report additional results for three datasets: (1) Fake Movies, Real Actors; (2) Fake Movies, Fake Actors; and (3) Real Movies, Real Actors (shuffled). We also include token-rank results for the Fake Movies, Real Actors dataset.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

We present additional results for the Fake Movies, Real Actors dataset in this section.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

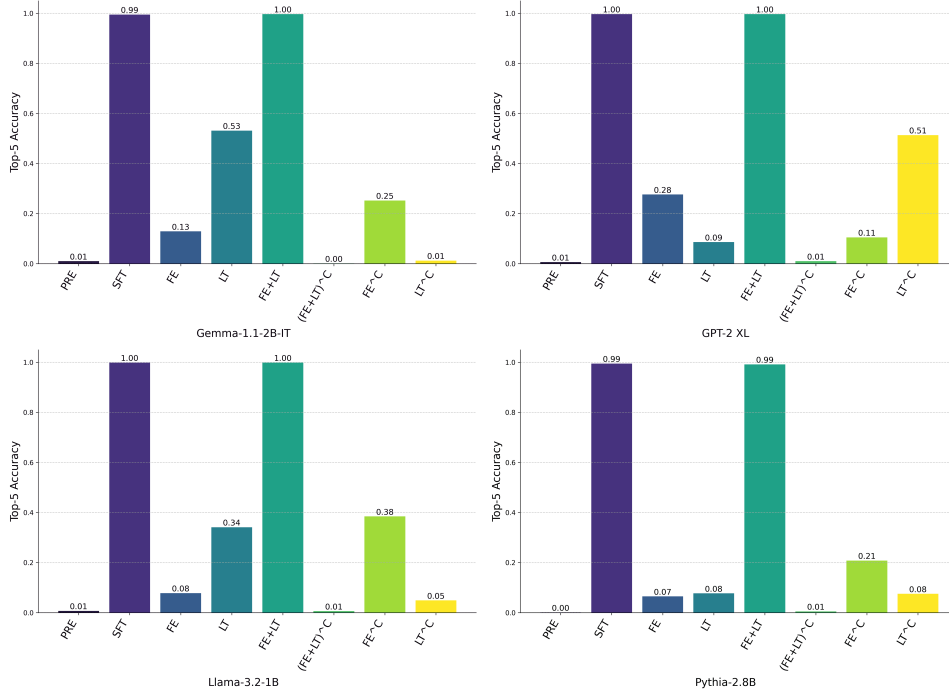


Figure 7: Top-5 accuracy — Sentence 1

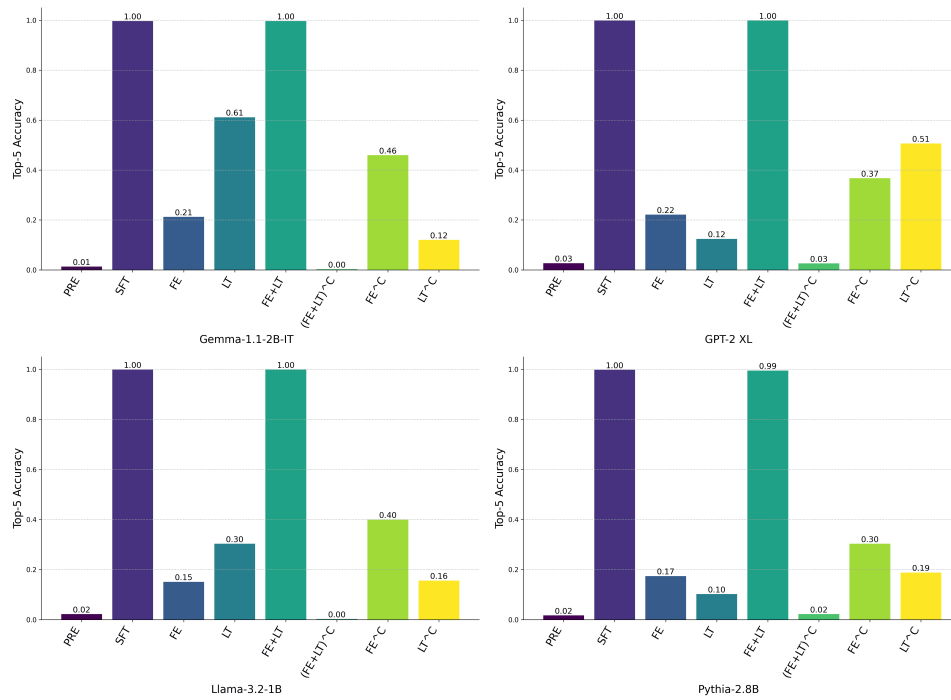


Figure 8: Top-5 accuracy — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

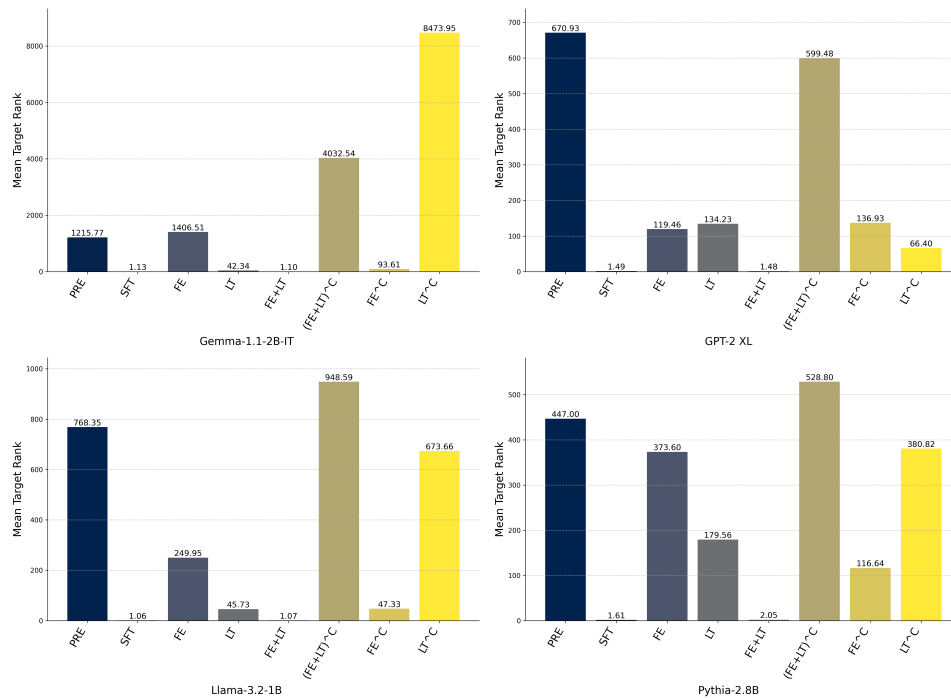


Figure 9: Mean target token rank — Sentence 1

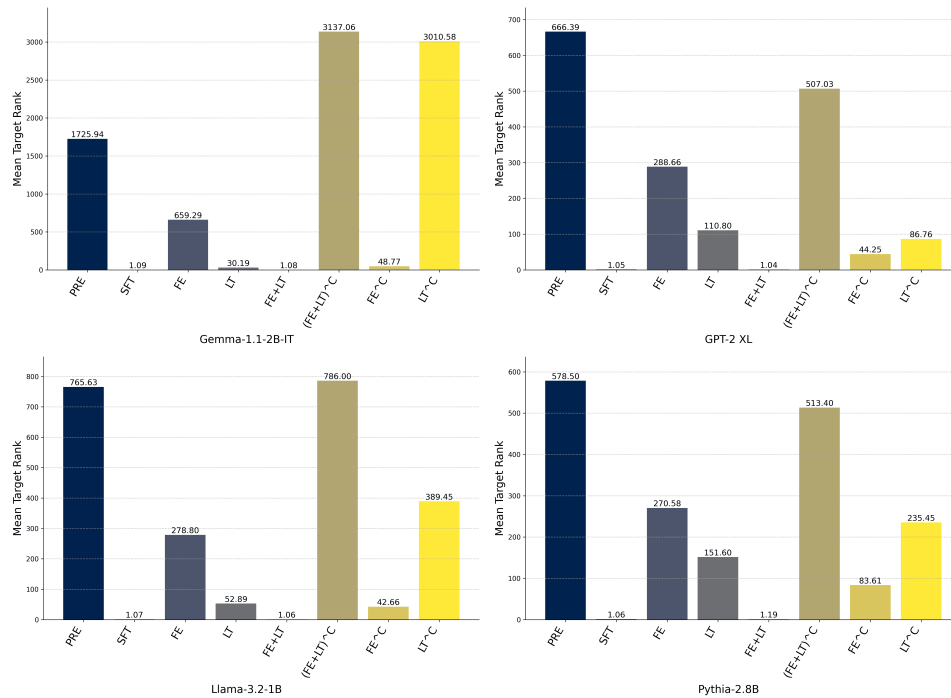


Figure 10: Mean target token rank — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

We also test additional values for k for the Fake Movies, Real Actors dataset. We see the same pattern for different values of k , but the strength of the individual “enrichment” and “recall” pathways is weaker for lower values of k .

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

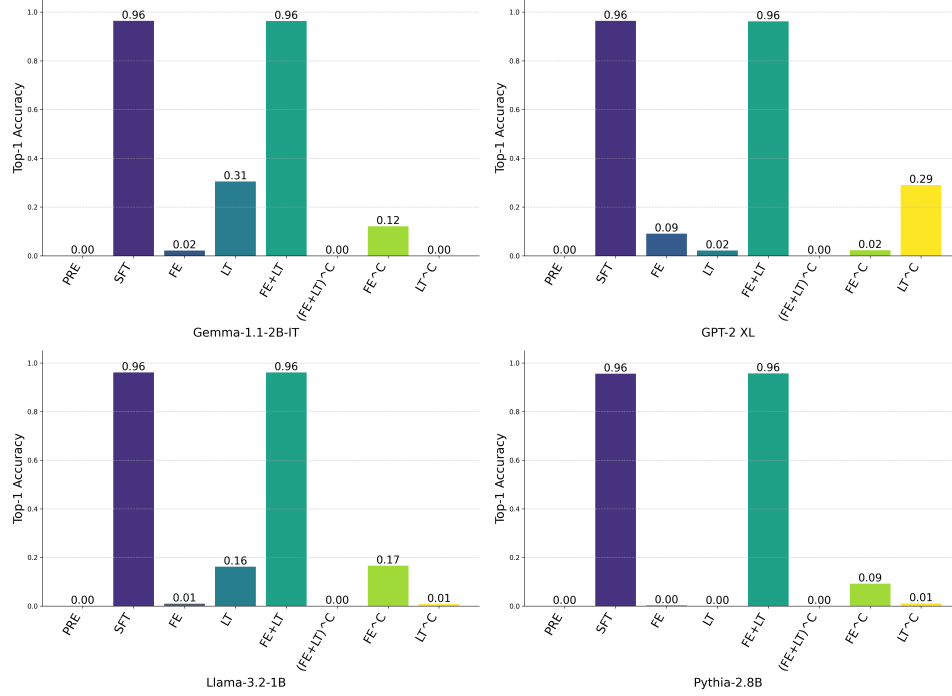


Figure 11: Top-1 accuracy — Sentence 1

C.2.2 TOP-10 RESULTS

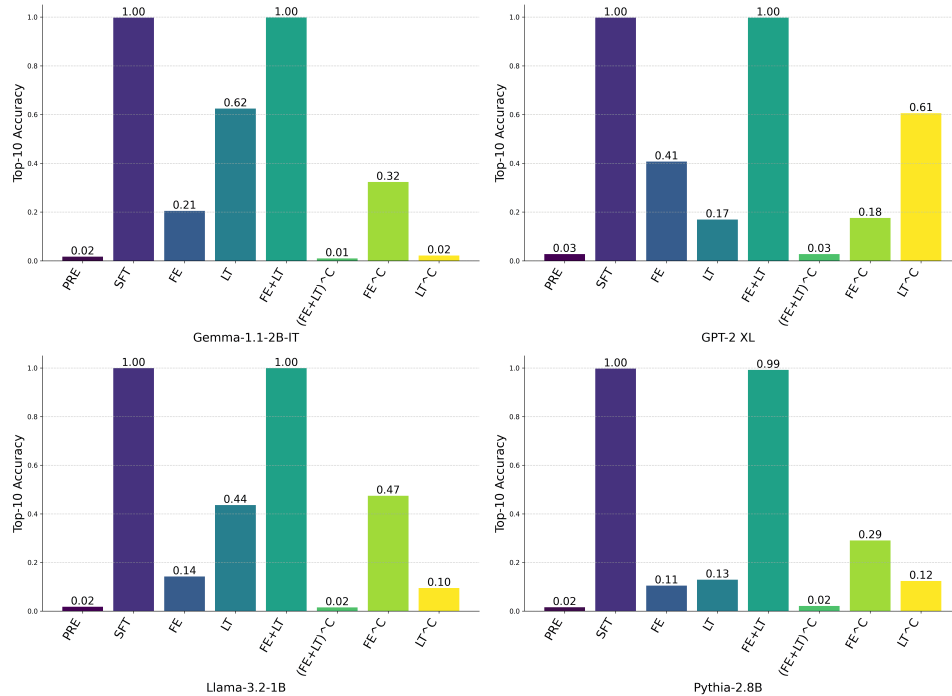


Figure 12: Top-10 accuracy — Sentence 1

C.2.3 TOP-100 RESULTS

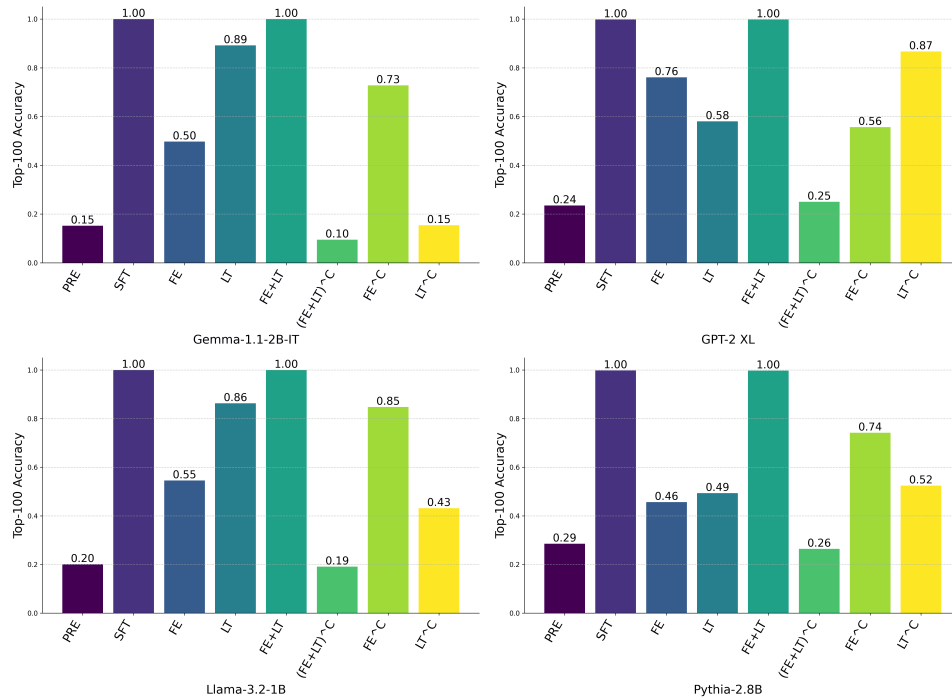


Figure 13: Top-10 accuracy — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

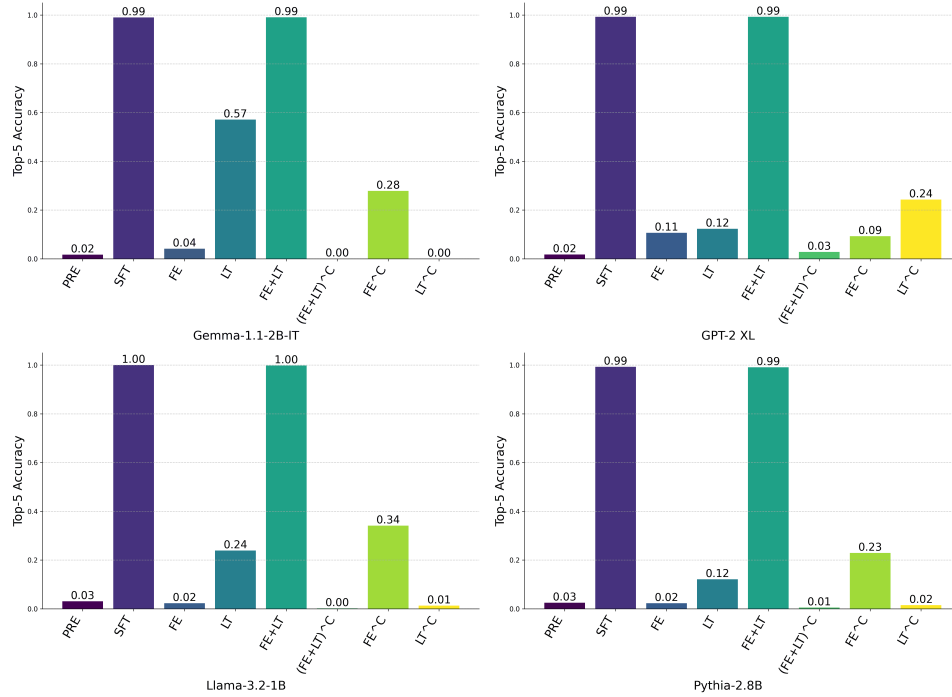


Figure 14: Top-5 accuracy — Sentence 1

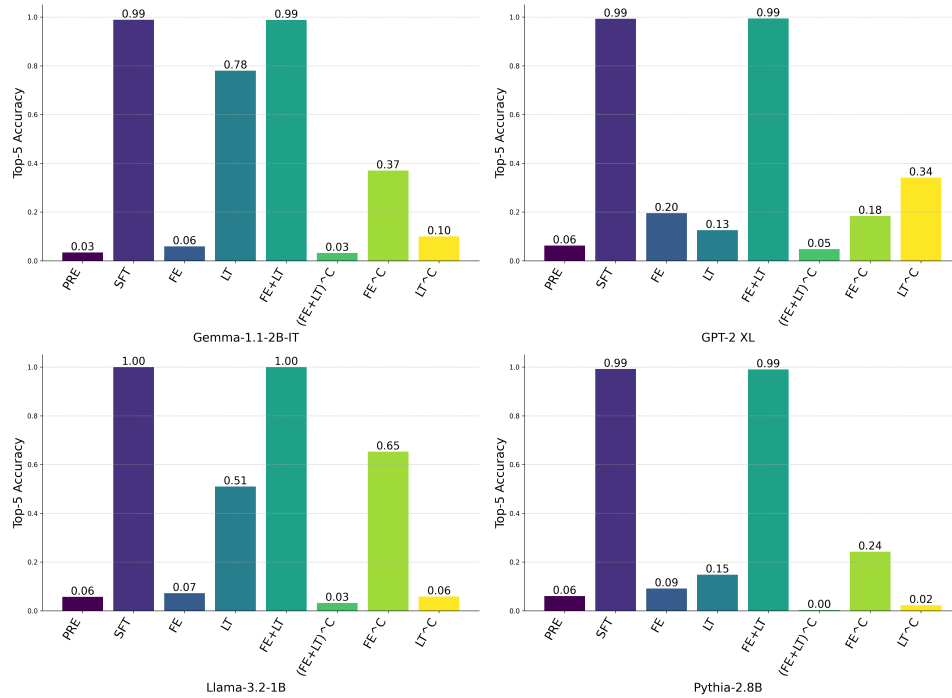


Figure 15: Top-5 accuracy — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

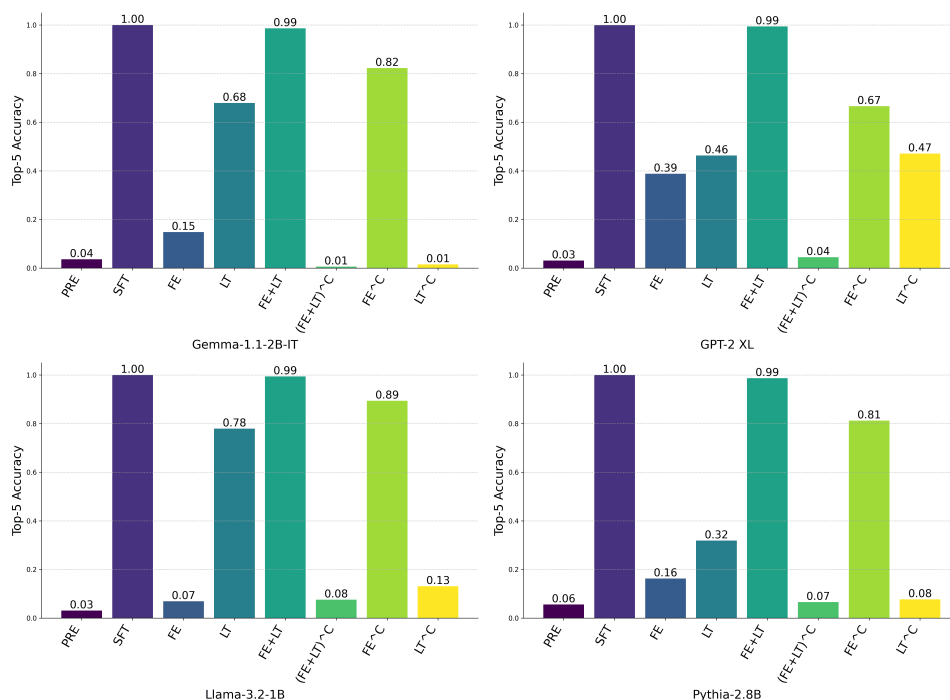


Figure 16: Top-5 accuracy — Sentence 1

C.5 MOVIE TITLE RESULTS

These experiments apply dynamic weight grafting with the movie title included in the test sentence. The movie title alone is insufficient to recover the correct entity, but combining the movie title with the final token improves performance for all models. The interaction between movie title and first entity is model-dependent: relative to the first entity alone, adding the movie title helps GPT-2 XL, has little effect for Llama 3 and Pythia, and hurts Gemma. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

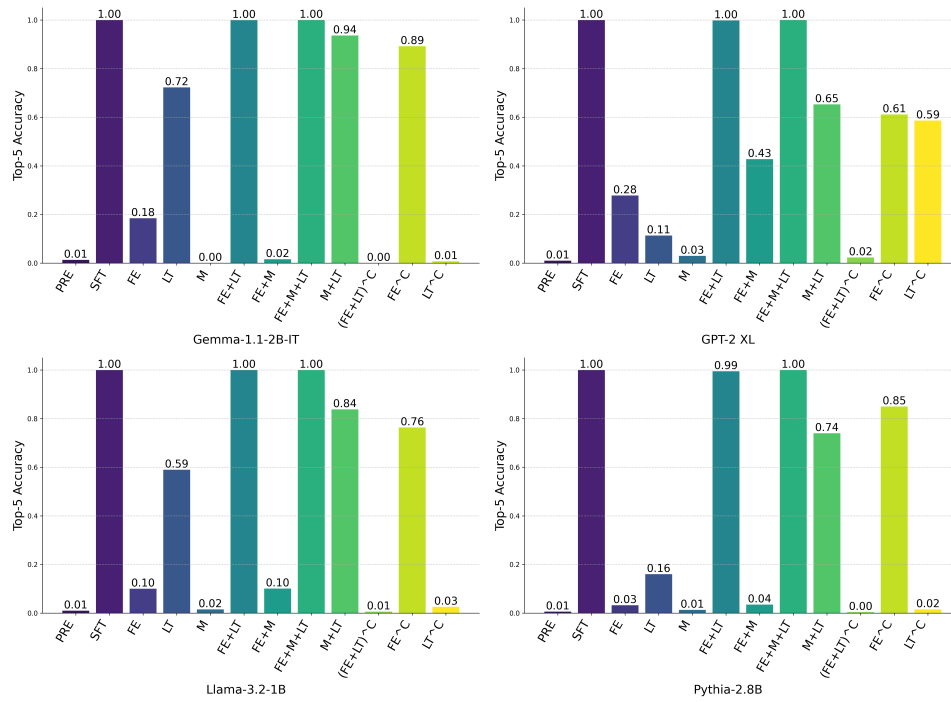


Figure 17: Top-5 accuracy–Sentence 3, where “M” denotes the movie title.

C.6 UNEMBEDDING MATRIX RESULTS

These results use finetuned unembeddings. Although top- k accuracy changes in some cases, especially for individual token positions, the overall pattern matches the results obtained with pretrained unembeddings.

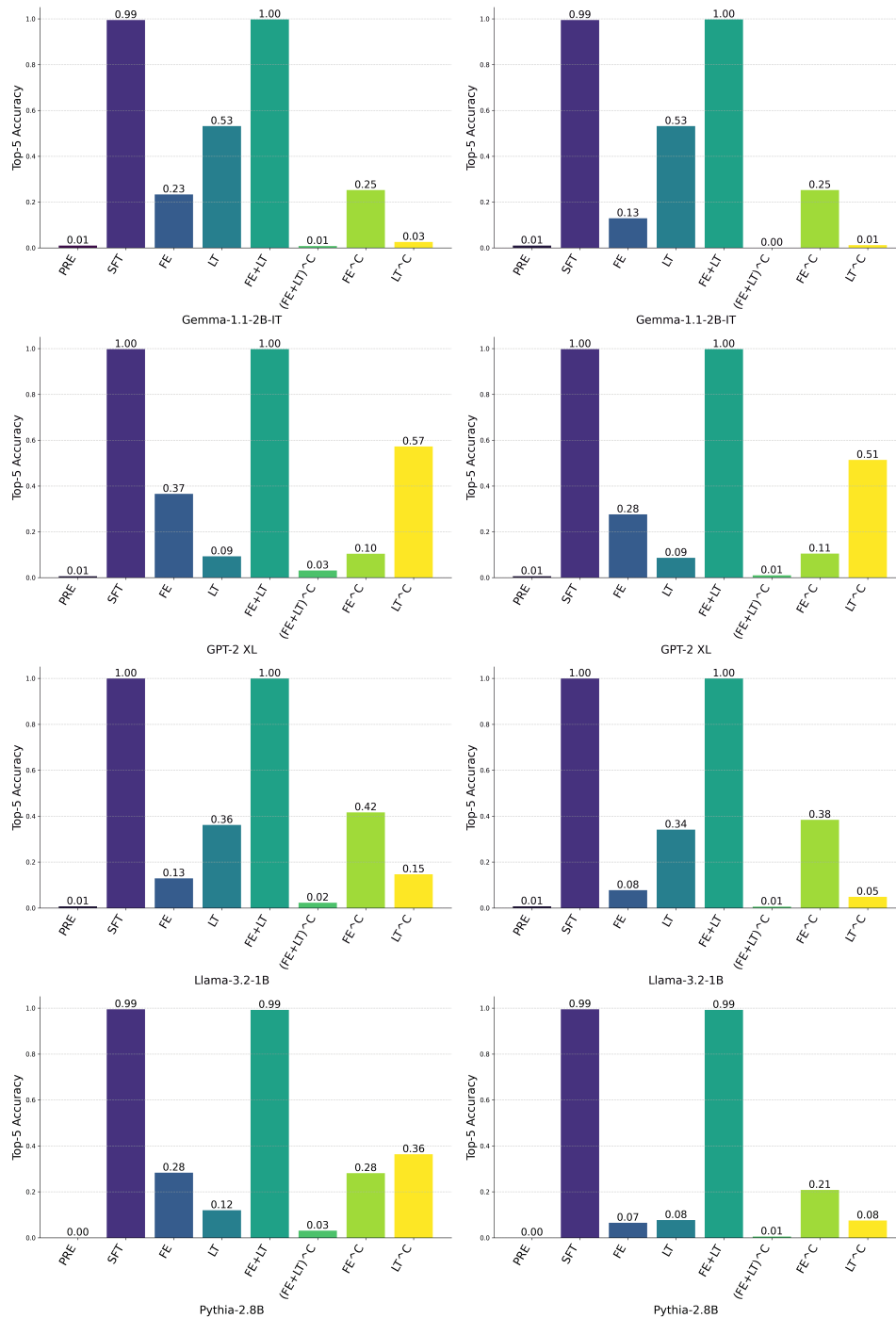


Figure 18: Top-5 accuracy–Sentence 1. Finetuned unembeddings are shown on the left and pretrained unembeddings on the right. The two settings produce similar patterns, although finetuned unembeddings yield higher top-5 accuracy for the first-entity-only condition.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports less aggressive finetuning experiments using a lower learning rate, zero weight decay, and supplemental training data from openwebtext and IMDB. The qualitative pattern remains similar, although the individual “extraction” and “recall” pathways are weaker.

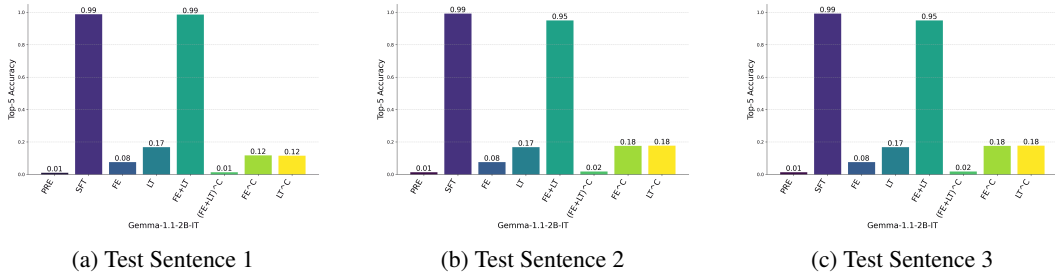


Figure 19: Top-5 accuracy for Gemma finetuned with a lower learning rate, zero weight decay, and supplemental training data from openwebtext and IMDB.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section provides baseline results for component-grafting experiments that graft from SFT models onto pretrained models.

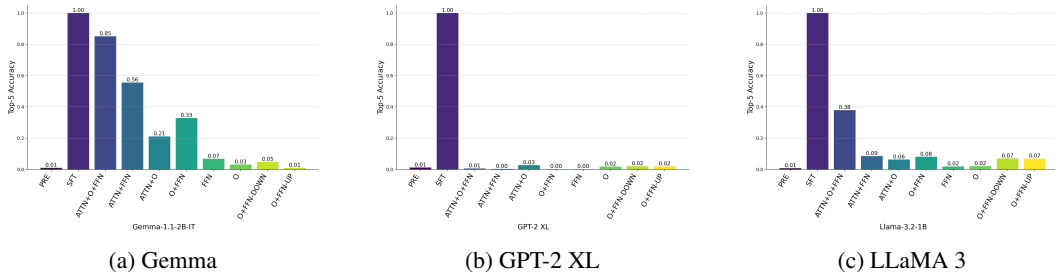


Figure 20: Top-5 accuracy for component-grafting baselines — Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

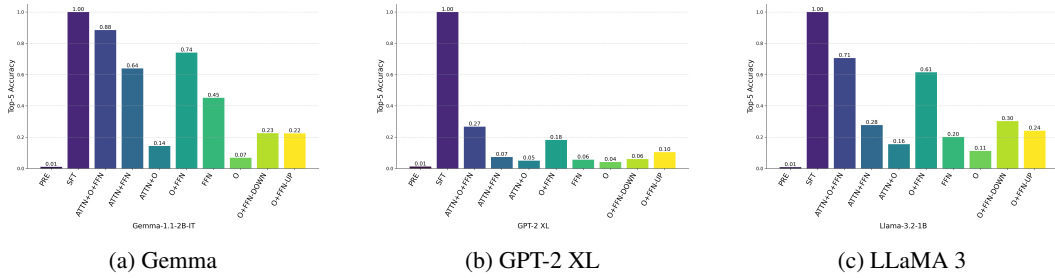


Figure 21: Top-5 accuracy for reversal-curse component-grafting results — Sentence 1

1566 C.10 TOKEN PROBABILITIES

1567
1568 We provide five randomly sampled token-probability outputs, each showing the top 10 tokens for
1569 representative experimental examples. Because the main evaluation uses top-5 accuracy, the top-10
1570 lists are included only to supply additional context.
1571

1572 C.10.1 LLAMA3: FE^C

1573
1574 Example 1:
1575 Target: Carolyn: 0.002
1576 J: 0.113
1577 L: 0.041
1578 Ch: 0.040
1579 Julie: 0.034
1580 Nicole: 0.024
1581 Nick: 0.023
1582 Michael: 0.023
1583 Jon: 0.023
1584 Jamie: 0.021
1585 Kelly: 0.017

1586 -----
1587 Example 2:
1588 Target: Lindsay: 0.024
1589 Michael: 0.093
1590 Ashley: 0.061
1591 Jennifer: 0.038
1592 Kim: 0.032
1593 Alexander: 0.025
1594 Lindsay: 0.024
1595 Milo: 0.023
1596 L: 0.021
1597 Jason: 0.018
1598 Lisa: 0.018

1599 -----
1600 Example 3:
1601 Target: Gwen: 0.015
1602 Michael: 0.030
1603 Paul: 0.028
1604 Julie: 0.024
1605 Elizabeth: 0.020
1606 Mary: 0.020
1607 S: 0.019
1608 E: 0.018
1609 J: 0.017
1610 L: 0.017
1611 A: 0.017

1612 -----
1613 Example 4:
1614 Target: Dominic: 0.006
1615 Is: 0.039
1616 Sarah: 0.031
1617 Jason: 0.026
1618 D: 0.026
1619 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1620 Mark: 0.019

1621

1622 -----

1623

1624 Example 5:

1625 Target: Ch: 0.045

1626 Ch: 0.045

1627 David: 0.024

1628 John: 0.022

1629 Peter: 0.018

1630 L: 0.018

1631 Mark: 0.018

1632 Michael: 0.017

1633 Richard: 0.016

1634 Ben: 0.012

1635 James: 0.012

1636

1637 -----

1638

1637 C.10.2 GEMMA: LT

1638

1639 Example 1:

1640 Target: Elizabeth: 0.040

1641 Julie: 0.495

1642 John: 0.130

1643 Stephen: 0.076

1644 Elizabeth: 0.040

1645 Hilary: 0.021

1646 Laurel: 0.018

1647 Tyne: 0.017

1648 Marian: 0.014

1649 Victoria: 0.013

1650 Juliet: 0.008

1651

1652 -----

1651 Example 2:

1652 Target: Uta: 0.008

1653 John: 0.901

1654 Joan: 0.022

1655 Sally: 0.011

1656 Chelsea: 0.009

1657 Uta: 0.008

1658 Tyne: 0.003

1659 Gina: 0.002

1660 Elle: 0.001

1661 Lisa: 0.001

1662 Rosemary: 0.001

1663

1664 -----

1664 Example 3:

1665 Target: Jennifer: 0.706

1666 Jennifer: 0.706

1667 John: 0.094

1668 Il: 0.028

1669 Me: 0.020

1670 Stephen: 0.015

1671 Carol: 0.012

1672 S: 0.008

1673 David: 0.008

1674 Elle: 0.007

1675 T: 0.006

```
1674 -----
1675
1676 Example 4:
1677 Target: Marcia: 0.017
1678 John: 0.152
1679 Gwen: 0.136
1680 Susan: 0.050
1681 Tim: 0.045
1682 Ch: 0.041
1683 Celeste: 0.037
1684 Tara: 0.030
1685 T: 0.028
1686 Elle: 0.028
1687 Brittany: 0.026
```

```
1688 -----
1689 Example 5:
1690 Target: Heather: 0.035
1691 Ly: 0.916
1692 Heather: 0.035
1693 Hilary: 0.011
1694 Stephen: 0.007
1695 Julie: 0.002
1696 Rosemary: 0.002
1697 Ch: 0.002
1698 Wanda: 0.001
1699 Chelsea: 0.001
1700 Tem: 0.001
1701 -----
```

1702 C.10.3 GPT-2 XL: (FE+LT)^C

```
1703
1704 Example 1:
1705 Target: Fre: 0.000
1706 her: 0.037
1707 John: 0.018
1708 Jason: 0.017
1709 Chris: 0.016
1710 Adam: 0.015
1711 Zach: 0.014
1712 Josh: 0.013
1713 the: 0.013
1714 Michael: 0.013
1715 Ben: 0.012
```

```
1716 -----
1717 Example 2:
1718 Target: A: 0.002
1719 her: 0.043
1720 the: 0.017
1721 John: 0.016
1722 Tom: 0.016
1723 Peter: 0.012
1724 fellow: 0.012
1725 Michael: 0.011
1726 David: 0.011
1727 James: 0.011
1728 Jack: 0.010
1729 -----
```

```
1728 Example 3:
1729 Target: Matthew: 0.003
1730 his: 0.057
1731 the: 0.017
1732 Jennifer: 0.014
1733 fellow: 0.012
1734 Chris: 0.011
1735 Michael: 0.011
1736 Jason: 0.009
1737 John: 0.008
1738 James: 0.008
1739 Jessica: 0.007
```

```
1739 -----
1740
1741 Example 4:
1742 Target: Cher: 0.000
1743 her: 0.042
1744 Tom: 0.026
1745 Robert: 0.019
1746 the: 0.018
1747 Matt: 0.018
1748 Michael: 0.016
1749 Brad: 0.016
1750 Chris: 0.014
1751 Johnny: 0.014
1752 John: 0.014
```

```
1752 -----
1753
1754 Example 5:
1755 Target: Timothy: 0.001
1756 his: 0.056
1757 the: 0.025
1758 Michael: 0.013
1759 fellow: 0.013
1760 Robert: 0.013
1761 John: 0.012
1762 James: 0.010
1763 Tom: 0.010
1764 Mark: 0.008
1765 Peter: 0.008
1766 -----
```

```
1766 -----
```

1767 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1768 A characteristic Gemma result under first-entity grafting is shown below: the model assigns high
1769 probability to common tokens while still placing the target token within the top five.
1770

```
1771 Target: Robert: 0.016
1772 a: 0.365
1773 an: 0.151
1774 the: 0.091
1775 Robert: 0.016
1776 two: 0.009
1777 his: 0.008
1778 other: 0.006
1779 John: 0.005
1780 Morgan: 0.005
1781 my: 0.005
```